

Deep-Bluffing: A Monte Carlo Expected-Value Heuristic Agent for Heads-Up Limit Texas Hold'em

Shorya Rathore

ICG Summer Project 2026–27 — Capstone Submission

Abstract

This report describes the design and validation of `CustomPokerBot`, an autonomous agent for Heads-Up Limit Texas Hold'em (HULHE) built entirely from scratch, without external poker libraries. The agent combines a Monte Carlo rollout estimator for hand-strength quantification with an expected-value (EV) pot-odds decision rule, layered with randomized bluffing, semi-bluffing, and slowplay frequencies to avoid being trivially exploitable. A key extension beyond a purely static strategy is an *opponent-adaptive bluffing* mechanism that infers, from indirect signals available in the arena API, how often the opponent folds, and scales bluff frequency accordingly. We also document a genuine bug discovered during edge-case hardening — an incorrect pot-odds calculation in short-stack / all-in scenarios — its fix, and its measured impact on performance.

1 System Overview

The submission is organized into four from-scratch modules:

- `cards.py` — a `Card` class with rank/suit parsing and total-ordering comparisons, plus a `make_deck()` helper.
- `hand_evaluator.py` — `evaluate_hand()`, which scores the best 5-card hand out of any 5–7 supplied cards by exhaustively checking all $\binom{7}{5} = 21$ combinations via `itertools.combinations`.
- `engine.py` — a full local HULHE simulator (blinds, fixed-limit betting, 4-bet cap, correct turn order, showdown resolution) used only for local testing/training; it is not part of the arena-facing submission.
- `agent.py` — the arena entry point, containing `CustomPokerBot(BasePokerBot)`, described in detail below.

Only `random`, `itertools`, and `collections` from the standard library are used, in compliance with the project’s “no external poker libraries” constraint.

2 State Representation

The arena API exposes a raw, unstructured state: two hole cards, zero to five community cards, integer pot size, stack size, and amount to call. Rather than hand-engineering a feature vector, we compress this state into a single scalar sufficient statistic:

$$\hat{p}_{\text{win}} = P(\text{our best 5-card hand} \succeq \text{opponent's best 5-card hand} \mid \text{known cards})$$

estimated via Monte Carlo rollout (Section 3). This is a deliberate design choice: instead of quantizing hand strength into discrete buckets (e.g. “top pair”, “draw”) via hand-crafted rules,

we let repeated random simulation of the unseen opponent hole cards and undealt board directly estimate the continuous quantity that actually matters for decision-making — our probability of winning the pot. All downstream logic (Section 4) is then a function of this single number, current pot odds, and a small number of hand-tuned frequency parameters for non-value play (bluffing, slowplay).

3 Monte Carlo Win-Probability Estimation

Given hole cards H and community cards B ($|B| \in \{0, 3, 4, 5\}$), let U be the set of unseen cards (the 52-card deck minus $H \cup B$). For $N = 300$ trials (reduced to 100–120 during stress-testing for speed), we:

1. Shuffle U and draw two cards as a hypothetical opponent hand H' .
2. Draw $5 - |B|$ further cards to complete the board, B' .
3. Evaluate both `evaluate_hand( $H \cup B \cup B'$ )` and `evaluate_hand( $H' \cup B \cup B'$ )` using the from-scratch evaluator.
4. Accumulate a win (1.0), tie (0.5), or loss (0.0) via `compare_hands`.

The estimator is

$$\hat{p}_{\text{win}} = \frac{1}{N} \sum_{i=1}^N (\mathbb{I}[\text{win}_i] + 0.5 \cdot \mathbb{I}[\text{tie}_i]).$$

This assumes a uniformly random opponent range, which is a simplification (real opponents narrow their range through betting), but is a standard and tractable baseline given the “no ML range models” scope of the project, and empirically produces strong decisions against both random and always-calling baseline opponents (Section 7).

4 Expected-Value Decision Logic

Standard pot-odds reasoning: calling costs c chips to potentially win a pot of size P . The break-even win probability required is

$$p_{\text{needed}} = \frac{c}{P + c}.$$

Calling is +EV whenever $\hat{p}_{\text{win}} > p_{\text{needed}}$, since

$$\text{EV}_{\text{call}} = \hat{p}_{\text{win}} \cdot P - (1 - \hat{p}_{\text{win}}) \cdot c.$$

4.1 The effective-call bugfix

A critical subtlety, found during edge-case hardening (Section 6), is that c is *not* always equal to the raw `amount_to_call`. The engine caps what a player actually pays at `min(amount_to_call, stack_size)` — an all-in call. The original implementation computed p_{needed} using the raw nominal `amount_to_call`, which badly overstates real risk whenever `amount_to_call > stack_size` (a short-stack / all-in-for-cheap spot). In the worst observed case (stack of 1 chip, nominal amount to call of 50, pot of 100), the bug caused the agent to **fold 100% of the time** on a call that, correctly evaluated, risks only 1 chip to win a pot of 101 ($p_{\text{needed}} \approx 0.01$) — an almost-always-correct call regardless of hand strength. The fix substitutes the effective, stack-capped cost:

$$c_{\text{eff}} = \min(\text{amount_to_call}, \text{stack_size}), \quad p_{\text{needed}} = \frac{c_{\text{eff}}}{P + c_{\text{eff}}}.$$

A secondary issue was found in the same pass: the “weak hand” decision branch (Section 4.2) used a flat 50% coin flip to decide whether to call whenever $p_{\text{needed}} < 0.15$, regardless of how favorable the odds actually were. This was replaced with a direct comparison $\hat{p}_{\text{win}} \geq p_{\text{needed}}$ as the primary decision, with randomized bluff-catching retained only for genuinely borderline cases. After both fixes, the same all-in scenario resolves as a call in every trial, and win rate against a random-action baseline opponent improved from 349W/146L/5T to 386W/109L/5T over identical 500-hand seeded matches (Section 7), confirming the bug was not confined to rare degenerate states but was suppressing legitimate calls broadly.

4.2 Decision tiers

Decisions are stratified into three win-probability tiers:

- **Strong** ($\hat{p}_{\text{win}} \geq 0.70$): value-raise with probability $1 - \text{SLOWPLAY_FREQ}$; otherwise call, to disguise hand strength.
- **Medium** (win probability between 0.45 and 0.70): follow pot odds, with a randomized semi-bluff raise (`SEMI_BLUFF_FREQ`) layered in on the stronger end of this band, for fold equity while retaining showdown value if called.
- **Weak** ($\hat{p}_{\text{win}} < 0.45$): call only when justified by c_{eff} -based pot odds (Section 4.1), fold otherwise, with a small randomized pure-bluff raise layer whose frequency is set adaptively (Section 5).

5 Opponent-Adaptive Bluffing

The arena API does not expose the opponent’s actions directly, only our own state at each decision point. To avoid bluffing at a fixed, exploitable frequency against opponents who never fold, we infer opponent foldiness from indirect signals available call-to-call.

5.1 Detecting hand boundaries

Hole cards change only when a new hand begins. Comparing `hole_cards` to the previous call’s value lets the agent detect hand boundaries without any explicit “hand ended” signal from the arena.

5.2 Inferring a fold from board progression

When the agent makes a pure or semi-bluff raise at community-card count $k = |B|$, it records k . At the start of the next hand, it compares k against the maximum board length subsequently observed, k_{max} , during the remainder of that hand:

$$\text{inferred_fold} = \begin{cases} \text{true} & \text{if } k_{\text{max}} = k \text{ (board never advanced)} \\ \text{false} & \text{if } k_{\text{max}} > k \text{ (opponent called into a new street)} \end{cases}$$

If the board never progresses past the street on which we bluffed, the hand must have ended immediately after our raise — since no further cards are ever dealt unless the opponent calls — so the opponent almost certainly folded. This signal is deliberately based on board progression rather than our own stack delta, because a stack increase alone conflates “opponent folded to our bluff”

with “we simply won at a legitimate showdown”; only one of those is evidence of foldability. The one case this cannot resolve is a bluff made on the river ($k = 5$, no further street to observe), which is skipped rather than guessed.

5.3 Exponential moving average and frequency scaling

The fold-rate estimate is updated with smoothing factor $\alpha = 0.15$:

$$\hat{f}_{t+1} = (1 - \alpha)\hat{f}_t + \alpha \cdot \mathcal{K}[\text{inferred_fold}], \quad \hat{f}_0 = 0.5 \text{ (neutral prior).}$$

The base bluff frequency $b_0 = 0.12$ is then scaled by $\hat{f}$ relative to the neutral midpoint:

$$\text{scale} = \begin{cases} 1 + \frac{\hat{f} - 0.5}{0.5}(s_{\max} - 1) & \hat{f} \geq 0.5 \\ 1 - \frac{0.5 - \hat{f}}{0.5}(1 - s_{\min}) & \hat{f} < 0.5 \end{cases}, \quad b = \text{clip}(b_0 \cdot \text{scale}, 0, 0.9)$$

with $s_{\min} = 0.15$, $s_{\max} = 2.0$. This produces near-zero bluffing against a pure calling station ($\hat{f} \rightarrow 0 \Rightarrow \text{scale} \rightarrow s_{\min}$) and up to double the base rate against a highly foldable opponent ($\hat{f} \rightarrow 1 \Rightarrow \text{scale} \rightarrow s_{\max}$).

5.4 Empirical effect

Against **AlwaysCallAgent** (never folds), the estimator correctly converges toward $\hat{f} \approx 0.2\text{--}0.35$ over a 500-hand match, suppressing bluff frequency accordingly. Because this opponent never folds, win/loss/tie *counts* cannot be affected by strategy (every non-folded hand reaches an equal, card-determined showdown) — this is a structural property of the opponent, not a shortcoming of the adaptation. What is affected is chip efficiency: an ablation with adaptation disabled (bluff frequency frozen at the neutral b_0) finished the identical seeded 500-hand match at 96 chips, versus 98 chips with adaptation enabled — adaptation measurably reduces chips wasted on bluffs that can never draw a fold against this opponent type.

6 Safety and Robustness

Because a single exception or illegal action zeroes the bot’s score in the arena, `get_action` wraps all logic in a `try/except` that falls back to `CALL` (or the first legal action) on any failure, and separately validates that the chosen action is a member of `legal_actions` before returning.

Beyond the randomized stress test in `validate_agent.py` (3,000 trials with uniformly random state), we added a targeted edge-case suite (`edge_case_test.py`) explicitly covering states that uniform random sampling under-visits: `stack_size` of 0, 1, 2, and 3; nominal `amount_to_call` exceeding `stack_size` (forced all-in calls); `amount_to_call` exactly equal to `stack_size`; zero pot size; single-element `legal_actions` lists; and river states with a near-exhausted deck. This suite is what surfaced the pot-odds bug described in Section 4.1.

7 Validation Results

8 Known Limitations and Future Work

- **Uniform opponent range assumption.** The Monte Carlo estimator assumes the opponent’s hole cards are uniformly random given the board, rather than conditioning on their

Table 1: Post-fix validation summary (`validate_agent.py`)

Check	Result
Signature match	PASS
Exceptions (3,000 random trials)	0 / 3,000
Illegal actions (3,000 random trials)	0 / 3,000
Avg. decision time	≈ 19.4 ms
Chip conservation (2,000 hands)	PASS (200 chips every hand)
Targeted edge-case suite (12 cases + repeats)	PASS

Table 2: Win rate vs. baseline opponents, 500-hand seeded matches

Opponent	Before fix	After fix	Final stacks (after fix)
RandomAgent	349W/146L/5T	386W/109L/5T	101 / 99
AlwaysCallAgent	237W/245L/18T	237W/245L/18T*	98 / 102

*Win/loss counts vs. AlwaysCallAgent are structurally determined by dealt cards, not strategy, since this opponent never folds — see Section 5. Chip efficiency (not shown here) still improves under adaptation, per the ablation in Section 5.

observed betting. A range-narrowing model (e.g. Bayesian updating from bet sizing) would improve estimate accuracy, particularly in later streets.

- **No fold equity vs. non-folding opponents.** By construction, no bluffing strategy can improve win/loss record against an opponent that calls with probability 1; this is a property of the opponent, not a solvable weakness of the agent.
- **River-bluff blind spot in opponent modeling.** Bluffs made on the river cannot be attributed to a fold vs. a call via the board-progression signal, since there is no further street to observe; these hands are excluded from the fold-rate update rather than guessed.
- **Simulation budget.** $N = 300$ rollouts balances accuracy against the ~ 20 ms per-decision budget observed during stress testing; a larger N would tighten the win-probability estimate at increased latency cost.

9 Conclusion

`CustomPokerBot` combines a from-scratch Monte Carlo win-probability estimator with pot-odds-based expected-value decisions, a randomized bluffing/slowplay layer for unpredictability, and an opponent-adaptive bluff-frequency mechanism that infers fold tendency from board-progression signals rather than raw stack deltas. Targeted edge-case testing surfaced and fixed a meaningful pot-odds calculation bug in short-stack scenarios, improving measured win rate against a random-action baseline from 349W/146L/5T to 386W/109L/5T over an identical seeded match, with the adaptive layer separately shown to reduce chip losses against a non-folding opponent even where it cannot change match outcomes.